Design Document for Session Management and Authentication for AgentCore Gateway

Proposed Solution 1

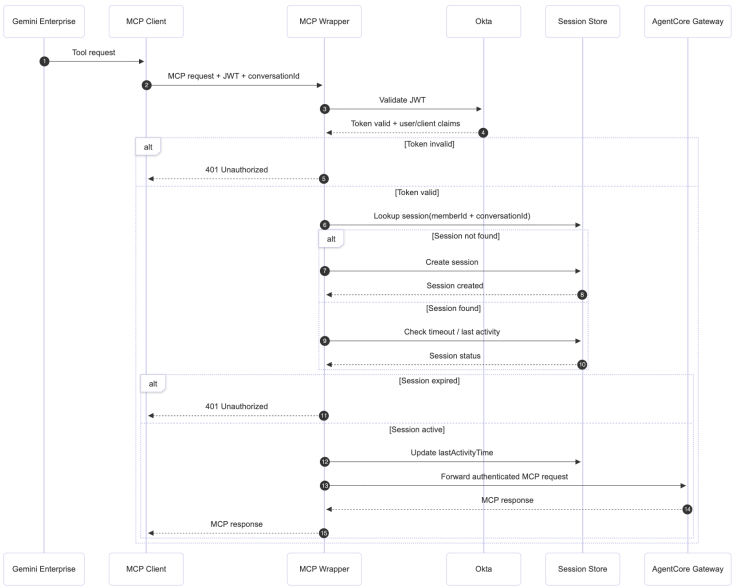
Overview

This document describes the design for implementing session management and JWT authentication validation for a conversational AI system in the insurance domain. The goal is to ensure that every request coming from Gemini Enterprise through the MCP client is authenticated and associated with a valid session before the request reaches the Orchestrator Agent and downstream BCBSM Agents

The solution introduces an MCP-based wrapper that combines both the authentication service and the Amazon Bedrock AgentCore Gateway functionality. This wrapper acts as a unified interface that first performs authentication and session validation before forwarding requests to the underlying gateway. Within this wrapper, a Python-based authentication service validates the Okta-issued JWT token, manages sessions keyed by `MemberID + conversationId`, enforces idle timeout rules, and detects token refresh events. Only after successful validation does the wrapper forward the request to the AgentCore Gateway, which then routes the request to the Orchestrator Agent and the downstream BCBSM Agents

Importantly, this design keeps the client interaction consistent. Gemini MCP interacts with the MCP wrapper as a single endpoint, while the wrapper internally coordinates authentication validation and gateway invocation.

Sequence Diagram



Design Objectives

The proposed solution is designed to achieve the following key objectives to ensure secure, reliable, and controlled access to the backend orchestration services.

- **Secure Authentication**

Every incoming request must be authenticated using Okta OIDC JWT access tokens, and the MCP wrapper verifies not only token validity but also that the token was issued for the expected client application: it performs local JWT checks for signature (via Okta JWKS), issuer, audience, expiration, and then enforces client binding by validating client-related claims such as cid (and optionally scopes) against an allowlist of approved app clients; for stronger assurance, high-risk flows can additionally require token introspection, and if available, proof-of-possession controls such as DPoP to confirm the calling app actually possesses the bound credential, so only requests with valid identity, valid client binding, and valid session context are treated as trusted and allowed to proceed.

- **Session Management**

The session management design uses Redis as a centralized, low-latency store where each conversation session is tracked with a key in the format MemberID :conversationId and fields such as lastActivity, tokenHash, and createdAt; on every request, the wrapper checks whether the session exists, creates it for first-time requests, validates that idle time has not exceeded the configured threshold for example 10 minutes, updates lastActivity for active sessions, and resets key TTL so inactivity automatically expires sessions, while also detecting refreshed tokens by hash change to keep session continuity without storing raw tokens**.**

- **Idle Timeout Enforcement**

To improve security and resource management, sessions are subject to an idle timeout policy. If a session remains inactive for more than 10 minutes, it is considered expired. Any subsequent requests associated with the expired session will be rejected, requiring a new authenticated session to be established.

- **Token Refresh Detection**

The system is capable of detecting when a client provides a refreshed JWT token during an active session. This is achieved by computing and comparing a hash of the incoming token with the previously stored token hash in the session record. If a change is detected, the session record is updated to reflect the new token while preserving the session continuity.

- **Gateway Protection**

A critical objective of the design is to ensure that only authenticated and valid sessions are permitted to reach the Amazon Bedrock AgentCore Gateway. The MCP wrapper performs all authentication and session validation checks before forwarding requests to the gateway,

effectively preventing unauthorized or expired requests from reaching the orchestration layer.